Acompáñame · Decisiones técnicas y proceso

Oct 7, 2026 · @Andre

1. Contexto

Acompáñame es un marketplace que conecta a familias que necesitan cuidados puntuales para una persona mayor o dependiente con cuidadores profesionales. Es el proyecto final del bootcamp de Desarrollo Web Full Stack (850 h) de Factoría F5, realizado en solitario.

El proyecto se compone de dos repositorios independientes: un **backend** (API REST con Spring Boot) y un **frontend** (aplicación Vue 3). La base de datos MySQL corre en Docker.

Aquí explico por qué tomé cada decisión, no solo lo que hice: qué opciones tenía, cuál elegí y qué he dejado para más adelante. Va junto a los README, los diagramas y el tablero de JIRA.

En esta entrega están el registro y el login, la búsqueda de cuidadores, las solicitudes con sus estados, las valoraciones y un pago de prueba. Lo demás (chat, geolocalización, videollamadas, recordatorios, informes en PDF y pago real) queda para una segunda fase, como cuento en la sección 9.

2. Arquitectura y stack

El backend sigue el patrón que vimos en clase: Controller, Service, Repository y Entity. Cada capa tiene una sola tarea: el controller recibe la petición, el service tiene las reglas del negocio y el repository habla con la base de datos. El código está agrupado por temas (usuario, solicitud, pago...) para tener junto lo que se toca a la vez.

Decisión	Por qué
Java 21 y Spring Boot	Es lo que trabajamos en el bootcamp, y ya trae seguridad y acceso a base de datos.
DTOs y Mappers	La API nunca devuelve las entidades tal cual. Así no se escapa la contraseña y el cliente solo puede enviar los campos que yo permito.
Interfaz genérica `InterfaceGenericService`	Varias entidades tienen el mismo CRUD; así escribo el contrato una sola vez.

Decisión	Por qué
Inyección por constructor	Es más limpia y facilita los tests con Mockito.
Excepciones propias y `GlobalExceptionHandler`	Cuando algo falla, la API responde con un código claro (403, 404, 409) y un mensaje sin datos internos.
Lombok	Evita escribir a mano getters, setters y constructores.
MySQL en Docker Compose	Se levanta con un solo comando y funciona igual en cualquier ordenador.
`ddl-auto=update`	No se borran los datos cada vez que reinicio la aplicación.
Vue 3 con `<script setup>`, Vite y Pinia	Menos código para escribir y arranque rápido.
CSS puro con BEM y variables	Control total del diseño sin añadir otra librería (ver sección 5).

Las credenciales de la base de datos no están escritas en el código: se leen de variables de entorno (DB_HOST, DB_USER, DB_PASSWORD…) desde un archivo .env que no se sube a Git. En el repositorio hay un .env.example como plantilla, y si no existe .env se usan los valores de desarrollo local.

3. Base de datos y modelo de datos

Seis entidades: `Usuario`, `Role`, `PerfilCuidador`, `Solicitud`, `Valoracion` y `Pago`.

Relación	Decisión
Usuario ↔ Role	`@ManyToMany` con los roles en **tabla propia**, no en un enum.
Usuario → PerfilCuidador	`@OneToOne`: solo los cuidadores tienen perfil profesional.
Solicitud → Usuario y PerfilCuidador	`@ManyToOne` en ambos: una familia hace muchas solicitudes y un cuidador recibe muchas.
Valoración → Solicitud	`@OneToOne`: se valora un servicio concreto y solo una vez, cuando está completado.

Relación	Decisión
Pago → Solicitud	`@OneToOne` con estados `PENDIENTE`, `COMPLETADO` y `CANCELADO`.

Los roles los guardo en una tabla y no en un enum. Un enum habría valido porque cada usuario tiene un solo rol, pero la tabla es lo habitual en Spring Security, permite que un usuario tenga varios roles y se pueden añadir roles nuevos sin tocar el código.

Usuario es una sola tabla. Pensé en separar los datos de login de los datos personales, pero para este proyecto solo lo complicaba. Si la app creciera, lo haría.

Una solicitud puede estar PENDIENTE, ACEPTADA, RECHAZADA o COMPLETADA. El estado se cambia con PATCH /solicitudes/{id}/estado. Uso PATCH y no PUT porque solo cambia un dato.

4. Seguridad

Decisión	Por qué
Basic Auth (email y contraseña)	Es lo más sencillo con Spring Security y es lo que dominaba. Sirve para demostrar login y permisos. JWT queda como mejora.
BCrypt	La contraseña se guarda cifrada y nunca se devuelve.
`JpaUserDetailsService` propio	Spring comprueba el login contra los usuarios reales de la base de datos, no contra usuarios de prueba.
`hasAuthority` en vez de `hasRole`	`hasRole` añade solo la palabra `ROLE_` delante. Con `hasAuthority` se compara el nombre exacto que hay en la base de datos.
Permisos por rol	Solo un cuidador puede crear, editar y borrar perfiles; solo una familia puede hacerlo con solicitudes.
Cada usuario solo toca lo suyo	Aunque alguien conozca el id de la solicitud de otra persona, la API le responde "acceso denegado". Esta comprobación está en el service. (Es lo que se llama IDOR.)
CORS solo para `http://localhost:5173`	La API solo acepta peticiones desde mi frontend.

Decisión	Por qué
Pocas rutas públicas	Solo registro, crear roles y login. Todo lo demás pide estar identificado.

Con Basic Auth el frontend guarda el email y la contraseña en sessionStorage para enviarlos en cada petición. Para una demo vale, y con JWT se mejoraría. También está pendiente cerrar POST /roles, que es público para poder cargar los roles al arrancar.

5. Frontend

Decisión	Por qué
Patrón Repository	Hay una clase por cada recurso (`SolicitudRepository`, etc.) que hace las llamadas al backend. Las pantallas no saben nada de `fetch` ni de la autenticación, y en los tests basta con simular el repositorio.
Pinia con `sessionStorage`	Guarda quién ha iniciado sesión. Si recargo la página sigue ahí, y se borra al cerrar la pestaña.
Vue Router	Una ruta por pantalla (13 en total), con acceso según haya sesión y según el rol.
Una sola `NavBar`	El menú cambia según seas Familia, Cuidador o visitante, sin repetirlo en cada pantalla.
`SolicitudCard` reutilizable	La misma tarjeta se usaba en dos pantallas.
BEM y variables CSS	Los colores y espacios están en un solo sitio; para cambiar la paleta toco un archivo.
Responsive a 768 px	Un solo punto de cambio entre móvil y escritorio, para mantener el CSS sencillo.
Pantallas vacías y mensajes de error	La app nunca se queda en blanco si algo falla o no hay datos.
Validar formularios	Compruebo los campos obligatorios antes de enviar. El backend valida también; la del frontend es para avisar rápido al usuario.

El pago es de prueba: la pantalla funciona completa y llama al backend con PATCH /pagos/{id}/pagar para marcar el pago como completado, pero los datos de la tarjeta son de prueba y no se cobra de verdad. Lo primero que haría después de la entrega es

conectarlo con una pasarela de pago real, por ejemplo Stripe en modo test.

6. Pruebas

Qué pruebo	Con qué	Para qué
Backend: servicios	JUnit 5 y Mockito	Probar la lógica de cada service sin base de datos, simulando los repositorios.
Backend: controladores	MockMvc y `@WebMvcTest`	Comprobar que cada endpoint devuelve el código y el JSON esperados.
Backend: integración	Testcontainers	Probar con una MySQL de verdad (en Docker) el repositorio de usuarios y la seguridad.
Frontend	Vitest, Vue Test Utils y Playwright	119 tests unitarios en 22 archivos, simulando los repositorios y sin necesitar el backend, y 9 tests e2e que recorren la aplicación completa en un navegador.

Uso Testcontainers y no una base de datos en memoria porque una base distinta de la real puede dar el test por bueno y luego fallar en MySQL. Tarda más y necesita Docker encendido, pero prueba lo que de verdad se usa.

Me centré en los casos límite: validaciones, errores de carga y que un usuario no pueda ver o tocar datos de otro.

Los tests de casos de error y de seguridad (403, 404, 409 y 400) los escribí al final y me sirvieron para encontrar dos fallos. Primero, si pedía un pago que no existía, la API daba un error 500 en vez de un 404; lo arreglé en el GlobalExceptionHandler. Segundo, cualquier usuario podía editar o borrar la valoración de otra persona; ahora se comprueba que sea la familia de la solicitud. Al final el backend tiene 179 tests y una cobertura del 96 % (JaCoCo); el frontend tiene 119 tests unitarios con un 86 % de cobertura (Vitest) y 9 tests e2e. En los dos se supera el 70 % mínimo que se pedía.

7. Diseño y gestión del proyecto

Primero hice bocetos, luego wireframes, mockups (móvil, tablet y escritorio) y un prototipo en Lovable. Así pude decidir pantallas, flujos y colores antes de programar, y después construir la app comparando con una referencia. Del prototipo cogí la paleta

verde con acento coral, la letra Nunito y la barra de navegación. En Figma están los wireframes y mockups.

Me puse una regla: si una pantalla no está en el diseño, no se hace. Por eso quité la pantalla de editar perfil del cuidador, que había añadido yo sin estar en el prototipo (más en la sección 8). Al final la recuperé para la entrega, ya con foto de perfil.

Para organizar el trabajo usé Scrum con JIRA (proyecto ACOM).

Elemento	Cantidad
Épicas	8
Historias de usuario (criterios en Gherkin)	19
Tareas técnicas	39
Sprints (agosto a octubre de 2026)	5

Las épicas son: Autenticación y Seguridad, Gestión de Cuidadores, Solicitudes de Servicio, Valoraciones, Pagos, Calidad y pruebas, Diseño y UX, e Infraestructura y entrega. Se eligió Gherkin (dado, cuando, entonces) porque convierte cada criterio de aceptación en algo directamente comprobable con un test.

En Git hice un commit por cada cambio, con los mensajes en inglés. Trabajé en main porque el proyecto es individual. Cuando el profesor me pidió una rama dev, la creé a partir de main y desde ahí sigo trabajando; al entregar paso el resultado a main.

Las historias HU-14 a HU-17 y las tareas T10 y T16 las he dejado en el backlog para después de la entrega.

8. Problemas encontrados y cómo se resolvieron

Problema	Qué pasaba	Cómo lo resolví
Los roles estaban escritos a mano en el registro	En el formulario puse los números de los roles (Familia = 4, Cuidador = 5). Si la base de datos cambiaba, el registro dejaba de funcionar.	El formulario ahora pide los roles al backend (`GET /roles`), así no depende de ningún número.
Los permisos por rol no se comprobaban como esperaba	Spring añade él solo la palabra `ROLE_` delante del nombre del rol y mis roles no la llevaban.	Uso `hasAuthority`, que comprueba el nombre tal cual está en la base de datos.

Problema	Qué pasaba	Cómo lo resolví
Seguridad con JWT sin haberlo visto en clase	Mientras desarrollaba, el profesor aún no había explicado JWT.	Empecé con Basic Auth, que sí dominábamos, y dejé JWT como mejora futura.
Cambios de última hora en la base de datos	A mitad de proyecto añadí el campo "vehículo propio" y la entidad Pago, y hubo que tocar la base de datos y rehacer los diagramas.	Actualicé la base de datos y los diagramas. Aprendí a cerrar el diseño de datos antes de programar.
Lovable no aplicaba todos los cambios que le pedía	Le mandaba una lista de mejoras y alguna se quedaba sin hacer.	Revisaba el resultado punto por punto y volvía a pedir lo que faltaba.
Estilos que no se aplicaban en el registro y la landing	Escribí `&--modificador` como en Sass dentro de un CSS normal. El navegador lo ignora sin avisar, y los botones y el rol elegido no cambiaban de aspecto.	Escribí cada selector completo (`.btn--primary`). Queda por revisar el resto de vistas.
Una pantalla de más: edición de perfil del cuidador	La construí sin que estuviera en el prototipo de Lovable.	La quité (ruta, vista y test, cada uno en su commit) y la dejé como mejora para después. Los tests pasaron de 32 a 27 y siguen en verde.
La app no se parecía al prototipo	Había construido desde los mockups sin comparar con Lovable (barra de navegación, fondo verde, iconos del registro).	Reescribí la barra de navegación, puse el fondo verde, cambié los iconos por SVG y oculté "Iniciar sesión" y "Registrarme" dentro de esas dos pantallas.
Los tests de integración fallaban	Necesitan Docker encendido para levantar la base de datos de pruebas.	Lo dejé escrito en el README: encender Docker antes de `./mvnw test` .

Problema	Qué pasaba	Cómo lo resolví
El Backlog de JIRA parecía vacío	La vista Backlog oculta lo que ya está en estado Listo. No era un fallo.	Lo comprobé en el cronograma y asigné las tareas que faltaban a los sprints 4 y 5.
Git marcaba `NavBar.vue` como modificado sin que hubiera cambios	Era solo una diferencia de saltos de línea entre Windows y Linux.	No hizo falta hacer nada; el contenido era el mismo.
Un pago que no existe daba error 500	Al escribir los tests de Pago vi que la API respondía "Error interno del servidor" en vez de 404. Faltaba su método en el GlobalExceptionHandler y el método genérico se lo tragaba.	Añadí el método que devuelve 404 y dos tests que lo comprueban. Lo aprendí: los tests de los casos de error sirven para encontrar fallos reales.

Lo que más he aprendido: comparar la interfaz con el prototipo desde el principio y no al final, y no añadir nada que no esté en el diseño.

9. Limitaciones conocidas, mejoras y fase 2

Limitaciones conocidas:

- Autenticación Basic Auth con credenciales en `sessionStorage`.
- Pago de prueba: la pantalla de pago llama al endpoint de pagos del backend y marca el pago como completado, pero la tarjeta es de prueba y no se cobra nada.
- Las credenciales ya se leen de variables de entorno, pero el valor por defecto de desarrollo sigue en `docker-compose.yml`.
- Algunas pantallas no coinciden al detalle con el prototipo de Lovable.
- Un único punto de corte responsive (768 px).
- Sin despliegue: todo corre en local.

Lo que haría a continuación, por orden:

1. Conectar el checkout con una pasarela de pago real, por ejemplo Stripe en modo test.
2. Migrar de Basic Auth a JWT.
3. Guardar las fotos de perfil como archivos externos, en lugar de dentro de la base de datos.
4. Desplegar frontend y backend, definiendo las variables de entorno en el servidor.

Para una segunda fase: chat en tiempo real, geolocalización de cuidadores, videollamadas, recordatorios y notificaciones, informes en PDF y pago real.

Se separó desde el principio lo que entra en esta entrega de lo que queda para esta fase, para terminar un producto completo y no uno a medias.